

Next.js Detailed Revision Notes (5-10 Pages)

1. Introduction

Next.js is a React framework that supports Server-Side Rendering (SSR), Static Site Generation (SSG), Incremental Static Regeneration (ISR), API routes, image optimization, routing, and full-stack development. It improves SEO, performance, and developer experience.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

2. Project Structure

The App Router uses the `app/` folder. `page.tsx` creates routes, `layout.tsx` provides shared layouts, `loading.tsx` shows loading UI, `error.tsx` handles errors, `not-found.tsx` customizes 404 pages, `route.ts` creates API endpoints, `globals.css` stores global styles.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

3. Server vs Client Components

Server Components render on the server by default and can directly fetch data, reducing bundle size. Client Components require `'use client'` and are used for state, event handlers, browser APIs, `useEffect`, and animations.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

4. Routing

Folders become routes automatically. Dynamic routes use `[id]`. Catch-all routes use `[...slug]`. Optional catch-all uses `[...slug?]`. Use `Link` from `next/link` for navigation and `useRouter()`, `usePathname()`, and `useSearchParams()` from `next/navigation`.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

5. Data Fetching

Server Components can be `async`. `fetch()` is extended by Next.js. `cache:'force-cache'` caches responses. `cache:'no-store'` disables cache. `next:{revalidate:60}` enables ISR by regenerating pages periodically.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

6. Rendering

SSR renders on every request. SSG renders during build time. ISR combines static generation with periodic updates. CSR renders entirely in the browser using Client Components.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

7. Server Actions

Server Actions allow forms and mutations without traditional API routes. Mark functions with `'use server'`. Useful for database inserts, updates, authentication, and form handling.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

8. API Routes

Create `app/api/users/route.ts`. Export GET, POST, PUT, DELETE, PATCH handlers. Return `Response.json()`. API routes are useful for backend logic, webhooks, and external integrations.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

9. Metadata & SEO

Use `export const metadata` or `generateMetadata()`. Configure title, description, Open Graph, Twitter cards, robots, icons, and canonical URLs to improve SEO.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

10. Images & Fonts

`next/image` automatically optimizes images with lazy loading and resizing. `next/font` downloads Google fonts at build time, improving Core Web Vitals.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

11. Middleware

`middleware.ts` runs before requests. Common uses include authentication, redirects, rewrites, localization, and protecting routes.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

12. Environment Variables

Store secrets in `.env.local`. Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser. Others remain server-only.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

13. Deployment

Next.js is optimized for Vercel but can run on Node.js servers. Use `npm run build` then `npm start`. Edge Runtime can improve latency for certain workloads.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

14. Interview Revision

Know differences between SSR, CSR, SSG, ISR; App Router vs Pages Router; Server vs Client Components; caching; revalidation; API routes; middleware; server actions; image optimization; metadata; dynamic routing.

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.

15. Cheat Sheet

page.tsx = route layout.tsx = shared layout loading.tsx = loading UI error.tsx = error boundary
route.ts = API endpoint 'use client' = client component 'use server' = server action next/link =
navigation next/image = optimized images revalidate = ISR

Example: Use practical examples in projects such as dashboards, e-commerce, blogs, authentication systems, and admin panels to understand where this feature fits.